

Dissertation dev log

Week 19:

Current Goals for Full Dissertation session:

- Research question made more precise and naturalistic
- Workplan make clearer
- Analyse data
- Specification for computing artefact that integrates societal, user, and business needs, demonstrating impact, usability and values to stakeholders
- Design rational, usage guidance, maintainability and architecture overview are provided
- Abstract
- Results and analysis section
- Discussions and conclusion
- Statistics addendum
- Artefact testing
- Critical addendum

Completed:

- Gather data
- A way to interact with the fragmented mesh
- A working Voronoi fracture in 2D for demonstration
- Working Voronoi fracture in 3D
- Comprehensive background done, covering triangulation, fracturing methods used in games, Graphics API's, Performance of Graphics API's, Point sampling, and Voronoi Clipping.
- Clearly stated research question
- Clear, feasible hypothesis, that is well scoped.
- An introduction that provides clear context for the project, and explains the need for the research, including positive impacts and who benefits. It should also highlight the motivation for the project.
- Professional, ethical, sustainable, legal, and social review carried out.
- A plan for security and privacy concerns has been written, and data is well managed.
- A methodology for how the project is going to be researched, and how its going to be evaluated
- A working way of profiling the results of the prototype
- Voronoi clipping implemented

- A Critical Review of current technology is completed, that highlights where the gap in research is, and the how this project fills it.
- Project is ethically robust
- Project plan with gantt chart
- Profiling results that are easily replicated and are well documented.
- Fracture implemented in both OpenGL and Vulkan
- Readme that highlights development roadmap
- OpenGL integrated into bridge system
- An implementation in GLSL, for performance comparison
-

Bugs

- OpenGL buffering of data has to be done in a specific order, i.e., declare positions, then buffer it, then declare textures, and then buffer them
- Mesh needs tetrahedralization, which means constrained Delaunay triangulation

Development notes:

Artefact is now fully functional and has been used to gather data. Data analysis has been started. 1000 points has been removed due to difficulty with thread temporary memory on the gpu being too small. However, this shouldn't impact the experiment much as 100 points is sufficient to test my hypothesis.

Images:

```

double TimeBeforeComputation = glfwGetTime();

voronoiCompute.Dispatch(NumGroupsX: points.GetSize(), NumGroupsY: 1, NumGroupsZ: 1);
voronoiCompute.WaitForCompletion();

TimeBeforeComputation = glfwGetTime() - TimeBeforeComputation;

if (points.GetSize() == 10)
{
    record.TenPointsGeneration = std::to_string(TimeBeforeComputation);
}
else if (points.GetSize() == 100)
{
    record.OneHundredPointsGeneration = std::to_string(TimeBeforeComputation);
}
else
{
    record.OneThousandPointsGeneration = std::to_string(TimeBeforeComputation);
}

UComputeShader clippingCompute = UComputeShader(InName: "VoronoiClipping", InStorageLocation: "/Shaders/Voronoi/");
//UComputeShader clippingCompute = UComputeShader("VoronoiClippingUnTriangulated", "/Shaders/Voronoi/");
clippingCompute.Use();
::DataBuffers::BindShaderStorageBuffer(VoronoiOut, Binding: 3, Size: sizeof(VOut));

::DataBuffers::BindShaderStorageBuffer(WBuffer, Binding: 6, Size: sizeof(WorkingBuffer));

::DataBuffers::GenerateShaderStorageBuffer(InTetsInd, Size: sizeof(InTets), Binding: 4);
::DataBuffers::GenerateShaderStorageBuffer(ClippedOutInd, Size: sizeof(VOutLarge), Binding: 5);
//::DataBuffers::GenerateShaderStorageBuffer(ClippedOutInd, sizeof(VOutUnTried), 5);

void* inTetPtr = ::DataBuffers::MapBufferMemory(InTetsInd, Size: sizeof(InTets));
InTets* inTetsData = static_cast<InTets*>(inTetPtr);
memcpy(inTetsData, &tets, Size: sizeof(InTets));

::DataBuffers::UnMapBufferMemory(InTetsInd);

TimeBeforeComputation = glfwGetTime();

clippingCompute.Dispatch(NumGroupsX: points.GetSize(), NumGroupsY: 1, NumGroupsZ: 1);
clippingCompute.WaitForCompletion();

TimeBeforeComputation = glfwGetTime() - TimeBeforeComputation;

if (points.GetSize() == 10)
{
    record.TenPointsClipping = std::to_string(TimeBeforeComputation);
}
else if (points.GetSize() == 100)
{
    record.OneHundredPointsClipping = std::to_string(TimeBeforeComputation);
}
else
{
    record.OneThousandPointsClipping = std::to_string(TimeBeforeComputation);
}

DataRecorder::SaveDataRecord(record, Name: "ExperimentData/CellGenerationResults.json");

```

References: